

Machine Learning and Deep Learning: Application

Jie Wang

17th Feb, 2025

Machine Learning: Classification and Evaluation

- Linear Regression
- Logistic Regression
- Softmax Regression
- Experimental Setting
- Other Paradigms
- Model Evaluation

Deep Learning: Basics

- Neural Network
- Transformer Architecture
- Applications

Deep Learning: Recent Research

- Pretrained Large Language Model
- Transfer Learning
- Reinforcement Learning and Transformers
- Applications

Machine Learning: Classification and Evaluation

Linear Regression

Predicting continuous values

Input: a set of labelled training data $T = (\mathbf{x}^{(i)}, y^{(i)})$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ (d-dimensional features, e.g., area=120m2, bedrooms=3), and $y^{(i)} \in \mathbb{R}$ (continuous target, e.g., price=\$520,000)

Output: A model vector $\hat{y}(x) = \mathbf{w}x$, $\mathbf{w}$ are the parameters

Prediction: $\hat{y}(x_{new}) = \mathbf{w}x_{new}$, $\hat{y} \in \mathbb{R}$

Loss: e.g., Mean Squared Error (MSE) $\min_{\theta} \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \mathbf{w}x^{(i)})^2$

Linear Regression

Examples

Predicting house prices based on the features.

Scenario: Predicting urban house prices.

Features: [Area=150 m², Bedrooms=2].

Trained Weights: $W = [W_0=10000, W_1=2000, W_2=50000,]\$$.

Prediction: Price= $10000+2000*150+50000*2=418,000$

Classification: Logistic Regression

Predicting probability

Input: a set of labelled training data $T = (\mathbf{x}^{(i)}, y^{(i)})$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ (d-dimensional features), and $y^{(i)} \in \{0, 1\}$ (binary classification, either 0 or 1)

Output: A probability model $\hat{y}(x) = \sigma(\mathbf{w}x)$, $\mathbf{W}$ are the parameters, $\hat{y} \in [0, 1]$

Prediction: $\hat{y}(x_{new}) = \begin{cases} 1 & \text{if } \sigma(\mathbf{w}x_{new}) > 0.5 \\ 0 & \text{otherwise} \end{cases}$

Loss: Cross entropy $\min -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$

Classification: Logistic Regression

Examples

Scenario: ICU Mortality Prediction.

Features: $x=[1, \text{Age}=72, \text{SaO2}=89\%, \text{Lactate}=4.2 \text{ mmol/L}]$

Trained Weights: $w=[-2.1, 0.03, -0.5, 1.2]$.

Prediction: $\sigma (-2.1+0.03 \times 72 - 0.5 \times 89 + 1.2 \times 4.2 = 0.784) = 0.687,$
 $\hat{y}=1$

Classification: Softmax Regression

Predicting Multi-class Classification

Input: a set of labelled training data $T = (\mathbf{x}^{(i)}, y^{(i)})$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ (d-dimensional features), and $y^{(i)} \in \{0, 1, \dots, k-1\}$ (k categories, from 0 to k-1)

Output: A probability model $\hat{y}(x) = \text{softmax}(\mathbf{w}x)$, $\mathbf{W}$ are the parameters

Prediction: $\hat{y}(x_{new}) = \text{the class with highest possibility}$

Loss: Cross entropy $\min -\frac{1}{n} \sum_{i=1}^n \sum_{k=0}^{K-1} (y^{(i)} = k) \log p(\hat{y} = k | x^{(i)})$

Classification: Softmax Regression

Examples

Scenario: Protein Function Prediction (k=20 classes)

Feature = 3D protein structure embedding, with d=256 dimension

Weight vectors Trained Weights: $W_{\text{kinase}} = [1.4, -0.2, \dots, 0.7]$, $W_{\text{GPCR}} = [-0.3, 1.1, \dots, 2.2]$

Prediction: $W_{\text{kinase}} = 5.3$, $W_{\text{GPCR}} = 3.8$

$\hat{y} = \text{kinase}$

Experimental Setting

Dataset Split

Training set (~ 80%): Model parameter optimization.

Validation set (~ 10%): Hyperparameter tuning (learning rate, regularization).

Test set (~ 10%): Final performance evaluation.

Training

Training involves feeding the model with input data and corresponding output data to adjust the model parameters.

Experimental Setting

Validation

Validation is the process of checking the model's performance on a separate dataset to ensure it generalizes well to unseen data.

Testing

Report evaluation results like accuracy, F1-score.

Other Paradigms

SVM

Maximizes margin between classes; kernel tricks for non-linearity.
Example: Image classification with high-dimensional data.

Model Evaluation

Accuracy

TP: True Positive

TN: True Negative

FP: False Positive

FN: False Negative

Accuracy = $\frac{TP + TN}{FP + FN + TP + TN}$

How many correct classifications in total.

Consider a set of predictions $\hat{y}_1, \dots, \hat{y}_N$ and a set of true labels $y_1, \dots, y_N$.

Mean accuracy is defined as: $A = \frac{1}{N} \sum (I(\hat{y}_i = y_i))$, $i = \{1, \dots, N\}$, $I(A)$ is 1 if A is true and 0 otherwise

Model Evaluation

Accuracy

Advantages:

- Can do binary or multi-class classification.

- Simple to compute.

- Single value

- Disadvantage: Doesn't take into account class imbalance.

Disadvantage: Doesn't take into account class imbalance.

- We're building a classifier to detect a rare disease.

- Assume only 1% of population is diseased. Diseased: $y = 1$ Healthy: $y = 0$

- What if we always predict healthy? ($y = 0$)

- Accuracy 99%

- But classifier is rubbish!

Model Evaluation

Precision and Recall

Precision: $TP/(TP + FP)$

The proportion of correctly identified diseased people from all our 'diseases' predictions.

The higher the better.

Recall: $TP/(TP + FN)$

The proportion of diseased people that are able to identify out of all the diseased people out there.

The higher the better.

Model Evaluation

F1 Score

Definition

Harmonic mean of Precision and Recall, balancing both metrics.

Formula

$$F1 = 2 * (Precision * Recall) / (Precision + Recall)$$

Model Evaluation

Sentiment Analysis

1. Task Objective

- **SemEval-2017 Task 4**: Sentiment Analysis in Twitter
- **Subtask A**: Message polarity classification
- Classes: Positive | Negative | Neutral

2. Dataset (IMDB Example)

```
from datasets import load_dataset  
dataset = load_dataset('semEval2017_task4')
```

Model Evaluation

Sentiment Analysis

Class distribution:

Train: 9,600 tweets | Test: 2,400 tweets

Class balance: 50% Neutral, 30% Positive, 20% Negative

Preprocessing steps:

1. Remove URLs/@mentions
 2. Handle emojis (preserve as text e.g., 😊 → :smile:)
- Expand contractions (don't → do not)

Model Evaluation

Sentiment Analysis

3. Model Comparison

Model Type	Example	Advantages	Limitations
Linear	Logistic Regression	Fast training	Needs manual feature engineering
Ensemble	RandomForest	Automatic feature interaction	Memory intensive
Probabilistic	Naive Bayes	Handles high dimensions	Strong independence assumptions
SVM	RBF Kernel	Good for small datasets	High computational complexity

Model Evaluation

Sentiment Analysis

4. Implementation Pipeline

Data Splitting

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    data['text'],
    data['sentiment'],
    test_size=0.2,
    stratify=data['sentiment'],
    random_state=42
)
```

Model Evaluation

Sentiment Analysis

Feature Engineering

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import FunctionTransformer

Text features
```

```
tfidf = TfidfVectorizer(
    ngram_range=(1,2),
    max_features=3000,
    stop_words='english'
)
```

Custom emoji counter

```
def count_emojis(text):
    emoji_pattern = re.compile("[
u\"\\U0001F600-\\U0001F64F\" # emoticons
u\"\\U0001F300-\\U0001F5FF\" # symbols
\"]+", flags=re.UNICODE)
    return len(emoji_pattern.findall(text))

emoji_counter = FunctionTransformer(lambda x: [count_emojis(t) for t in x])
```

Model Evaluation

Sentiment Analysis

Model Validation

```
from sklearn.pipeline import FeatureUnion, Pipeline
from sklearn.ensemble import RandomForestClassifier
```

Feature union

```
feature_union = FeatureUnion([
    ('text', tfidf),
    ('emoji', emoji_counter)
])
```

Complete pipeline

```
pipeline = Pipeline([
    ('features', feature_union),
    ('clf', RandomForestClassifier(class_weight='balanced'))
])
```

Cross-validation

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(pipeline, X_train, y_train,
    cv=5, scoring='f1_macro')
print(f'Average F1: {scores.mean():.3f}')
```

Model Evaluation

Sentiment Analysis

5. Evaluation Results

****Metrics**:**

```
from sklearn.metrics import classification_report
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
print(classification_report(y_test, y_pred,
target_names=['negative', 'neutral', 'positive']))
```

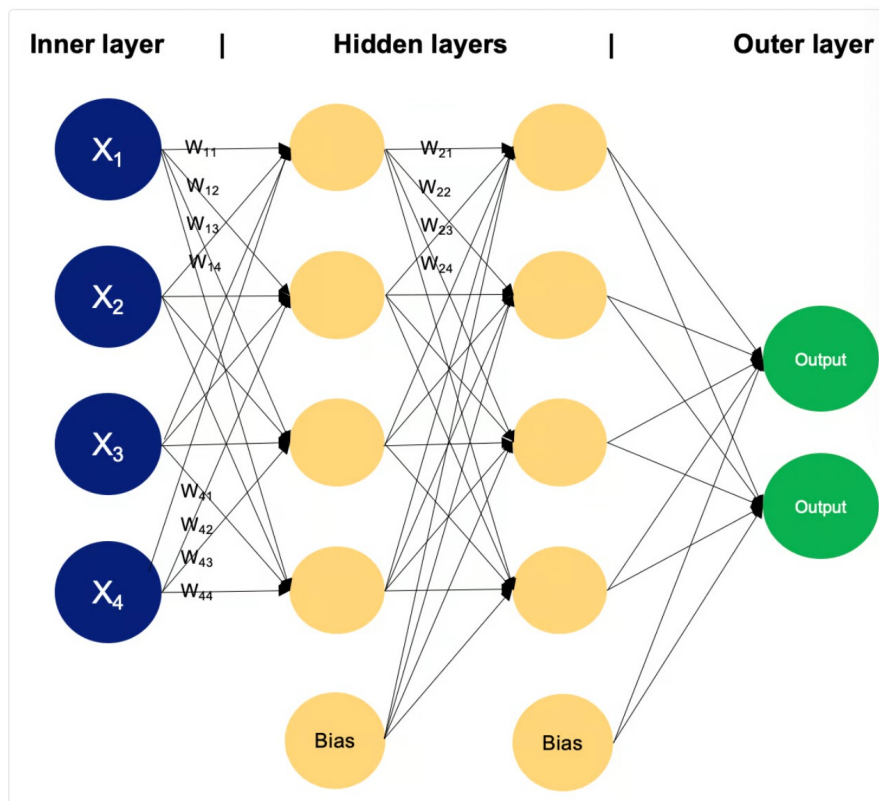
****Sample Output**:**

```
precision recall f1-score support
negative 0.68 0.62 0.65 423
neutral 0.72 0.75 0.73 672
positive 0.79 0.81 0.80 589
accuracy 0.74 1684
macro avg 0.73 0.73 0.73 1684
weighted avg 0.74 0.74 0.74 1684
```

Deep Learning: Basics

Neural Network

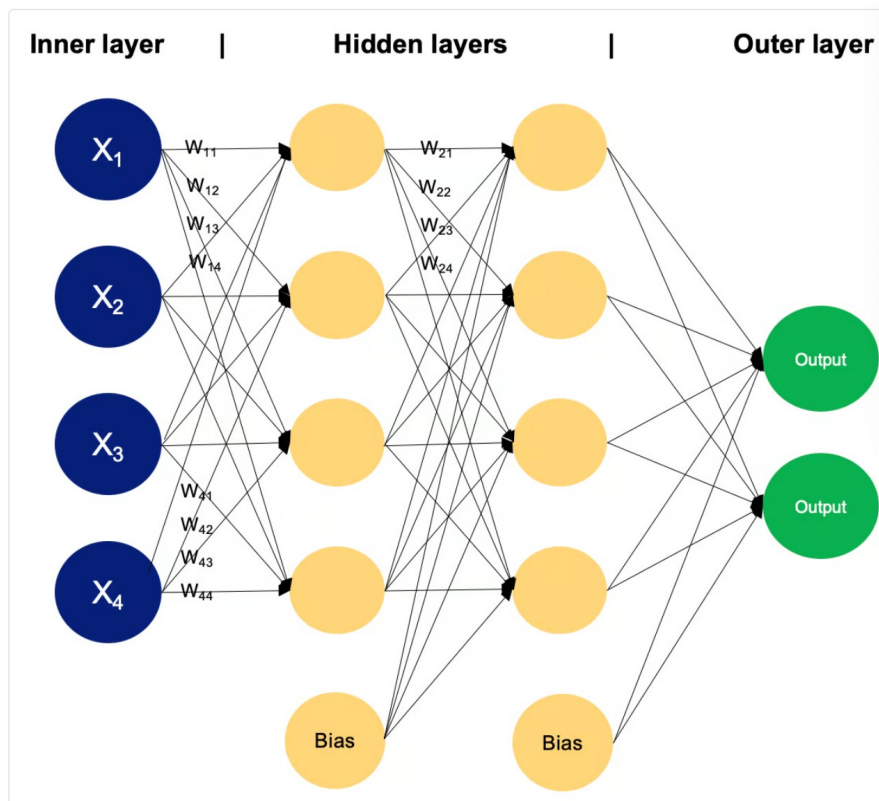
Multilayer perceptrons (MLP)



NN is a machine learning model inspired by the structure and function of the human brain's interconnected network of neurons. It consists of interconnected nodes called artificial neurons, organized into layers. Information flows through the network, with each neuron processing input signals and producing an output signal that influences other neurons in the network.

Neural Network

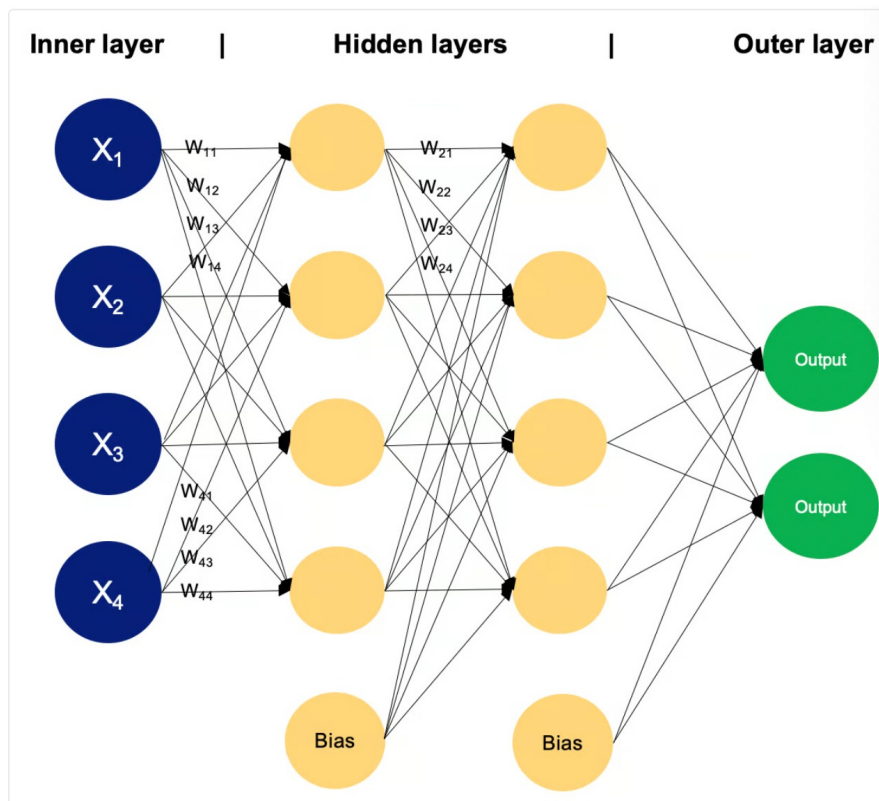
Multilayer perceptrons (MLP)



A MLP is a type of NN consisting of multiple layers of neurons. The neurons in the MLP typically use nonlinear activation functions, allowing the network to learn complex patterns in data. MLPs are significant in machine learning because they can learn nonlinear relationships in data, making them powerful models for tasks such as classification, regression, and pattern recognition. In this tutorial, we shall dive deeper into the basics of MLP and understand its inner workings.

Neural Network

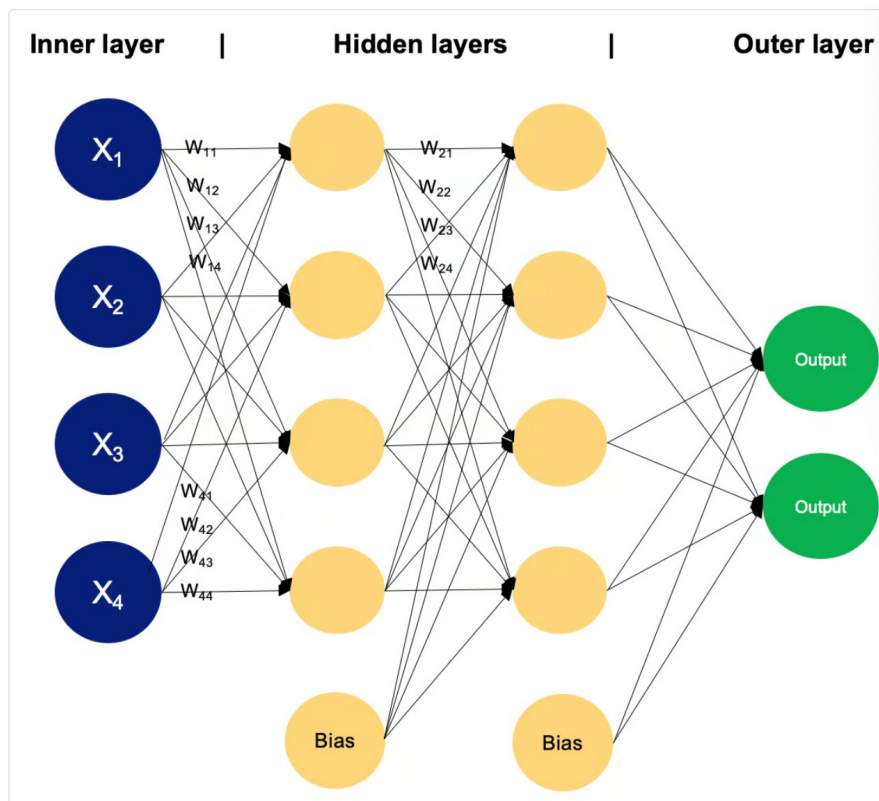
Multilayer perceptrons (MLP)



Forward Pass: During the forward pass, input data is fed into the neural network, and the network's output is computed layer by layer. Each neuron computes a weighted sum of its inputs, applies an activation function to the result, and passes the output to the neurons in the next layer.

Neural Network

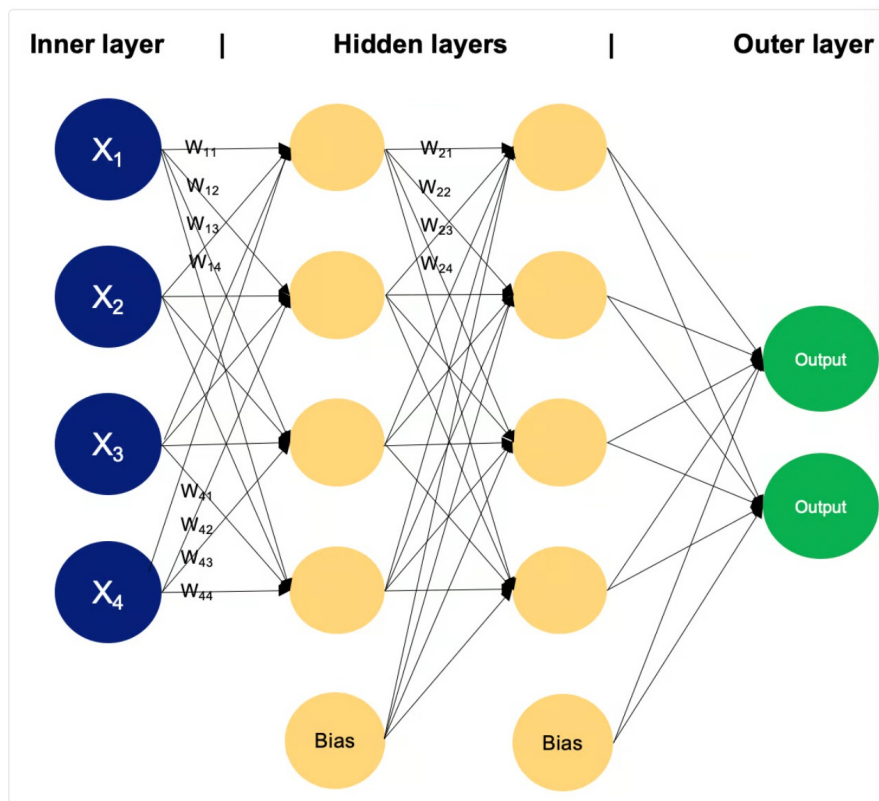
Multilayer perceptrons (MLP)



Loss Computation: After the forward pass, the network's output is compared to the true target values, and a loss function is computed to measure the discrepancy between the predicted output and the actual output.

Neural Network

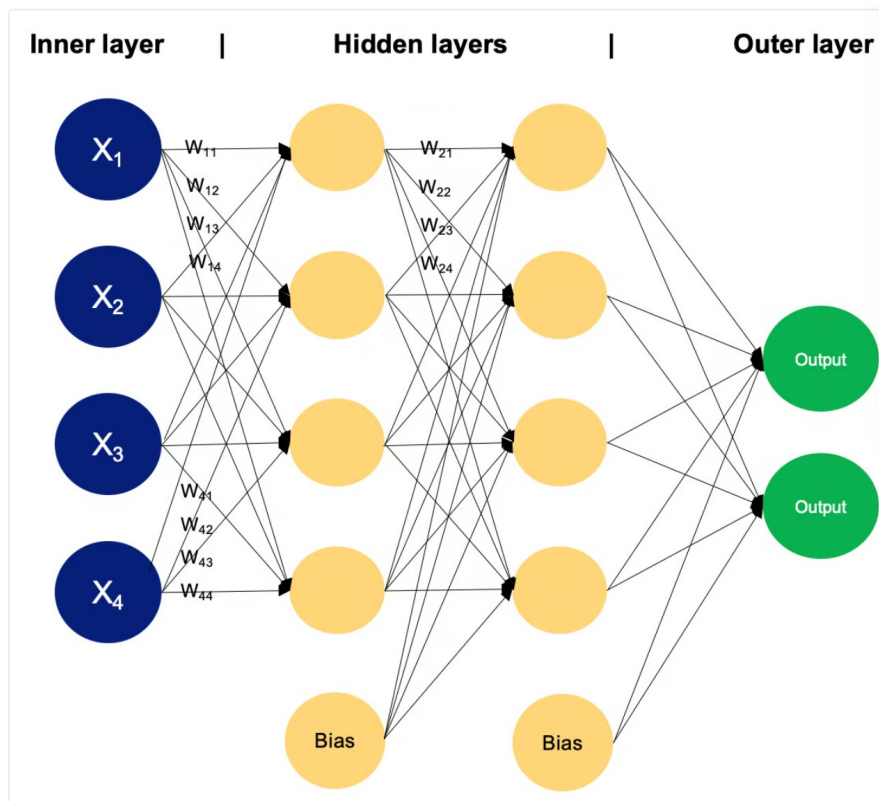
Multilayer perceptrons (MLP)



Backward Pass (Gradient Calculation): In the backward pass, the gradients of the loss function with respect to the network's parameters (weights and biases) are computed using the chain rule of calculus. The gradients represent the rate of change of the loss function with respect to each parameter and provide information about how to adjust the parameters to decrease the loss.

Neural Network

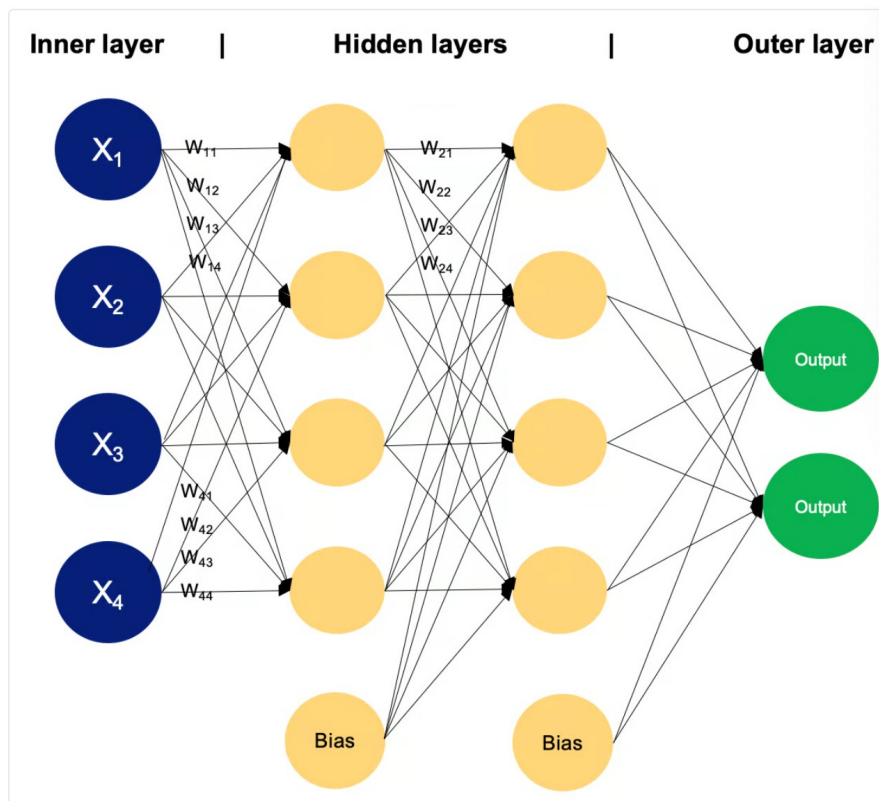
Multilayer perceptrons (MLP)



Parameter update: Once the gradients have been computed, the network's parameters are updated in the opposite direction of the gradients in order to minimize the loss function. This update is typically performed using an optimization algorithm such as stochastic gradient descent (SGD), that we discussed earlier.

Neural Network

Multilayer perceptrons (MLP)



Iterative Process: Steps 1-4 are repeated iteratively for a fixed number of epochs or until convergence criteria are met. During each iteration, the network's parameters are adjusted based on the gradients computed in the backward pass, gradually reducing the loss and improving the model's performance.

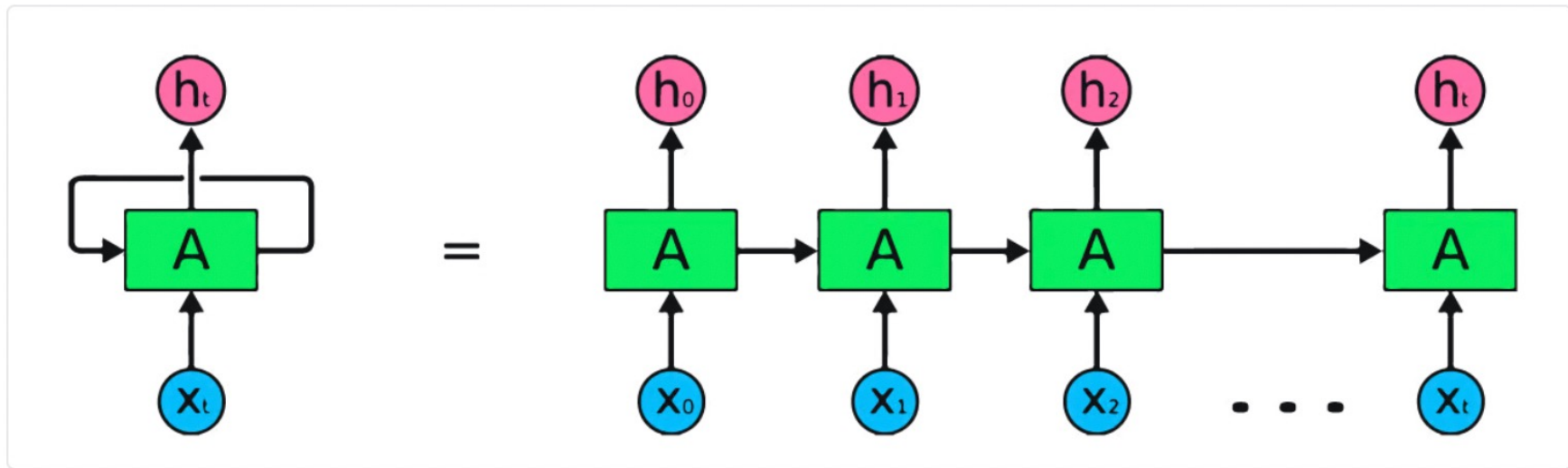
Neural Network

Classic methods

Recurrent Neural Networks(RNN), Long Short-Term Memory(LSTM), Convolutional Neural Network(CNN), ...

Neural Network

Classic Methods



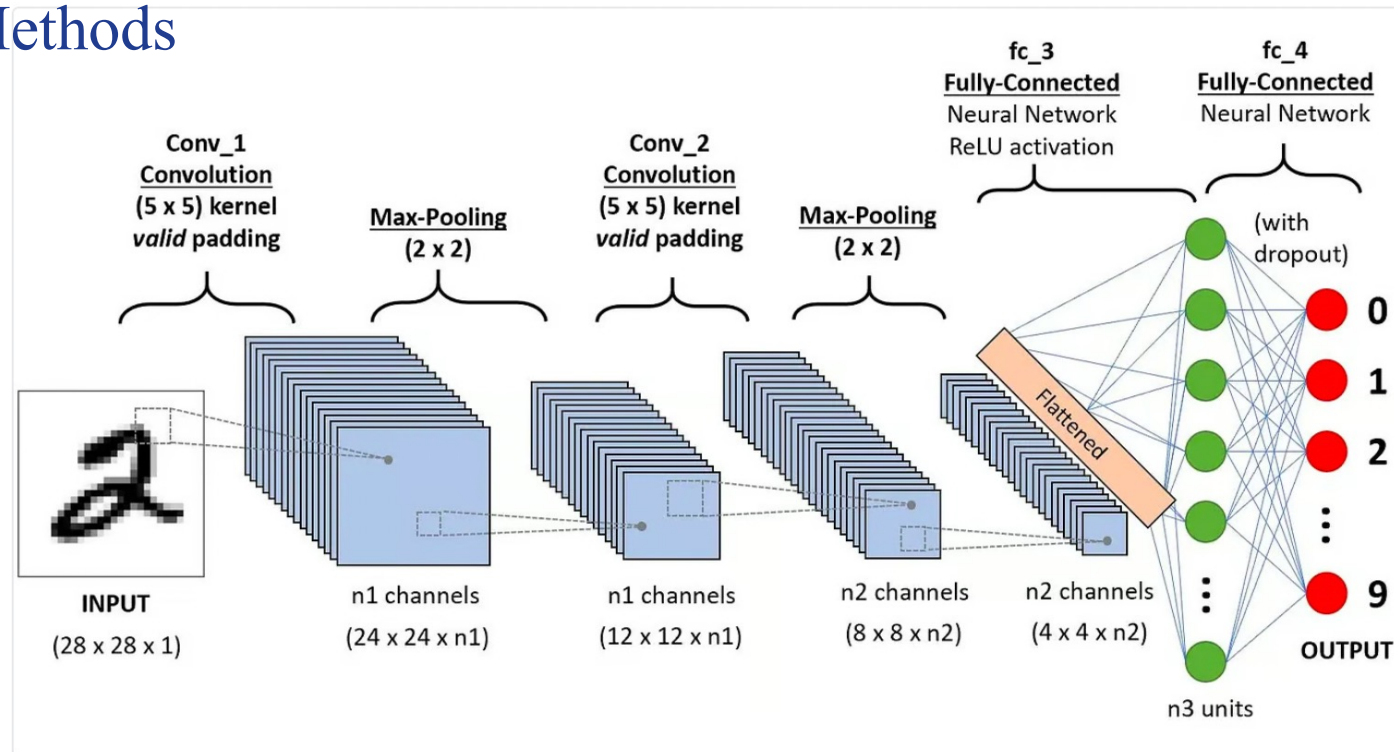
RNN

RNN: sequential data

Connections between nodes form directed cycles, allowing information to persist over time. This makes them suitable for tasks involving sequential data, such as time series prediction, natural language processing, and speech recognition.

Neural Network

Classic Methods



CNN: image data

Designed to effectively process grid-like data, such as images. They consist of layers of convolutional filters that learn hierarchical representations of features within the input data.

CNNs are widely used in tasks like image classification, object detection, and image segmentation.

Transformer

Transformer basics: Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

to compute Attention for all input tokens at once use Matrix operations (same as) where, $Q = XW^Q$, $K = XW^K$, and $V = XW^V$

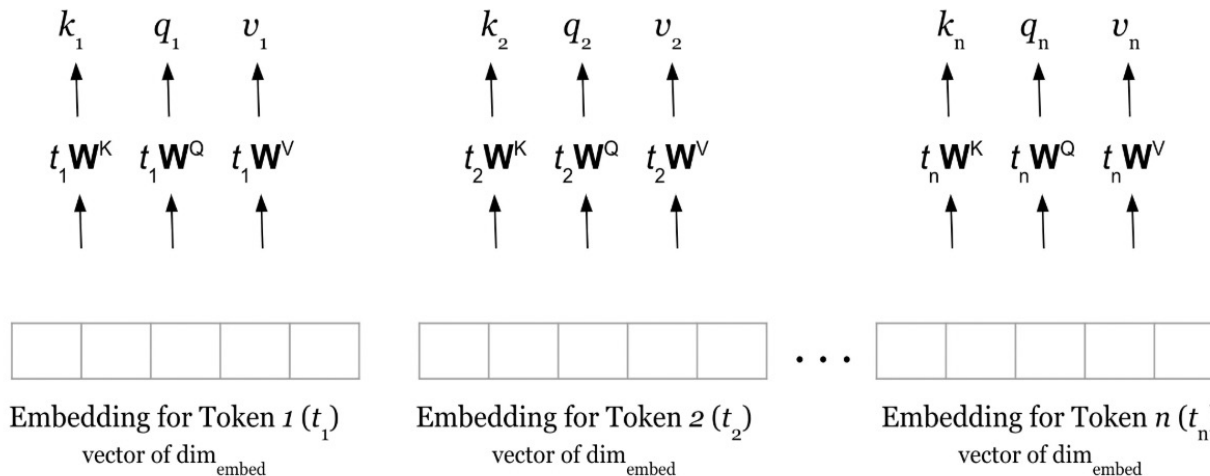
$$w_1 = k_1 \cdot q_n$$

$$w_2 = k_2 \cdot q_n$$

$$w_n = k_n \cdot q_n$$

$$\text{Attention}_{\text{token}_n} = \sum w_i v_i$$

© AIMA.com Research



④ Attention for token n is computed as a weighted sum of values of all the input tokens

③ dot product of key vector (k_i) of every input token with the query vector of n^{th} token (q_n) is used to dynamically compute weight (w_i) of each input token

② For each head, we initialize three matrices of $\text{dim}(\text{embed}, \text{head_size})$ called Key (W^K), Query (W^Q), and Value (W^V)

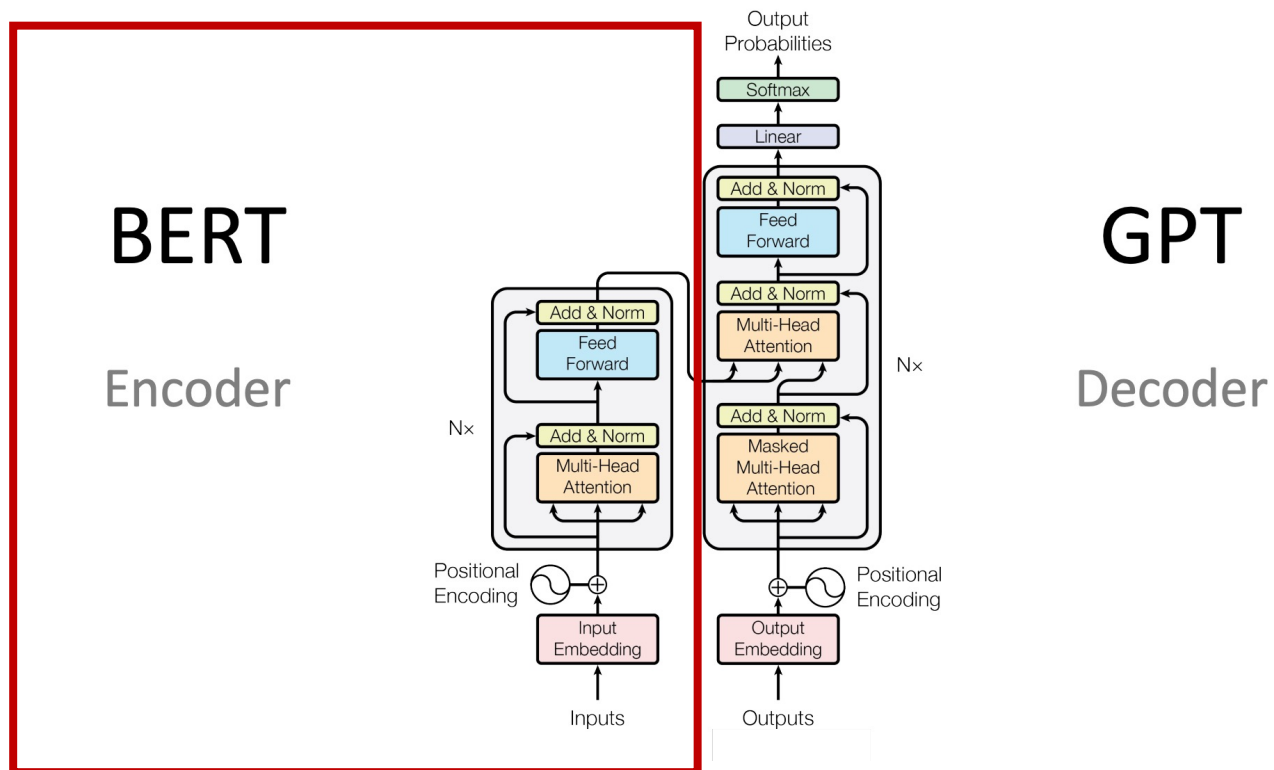
Embedding for each of the input token is multiplied by W^K , W^Q , and W^V matrices to generate k , q , and v vectors...

① Input is represented as matrix X of $\text{dim}(n, \text{embed})$, where n is the num of input tokens

Title: Step-by-step explanation of Self-Attention mechanism in Transformer Blocks

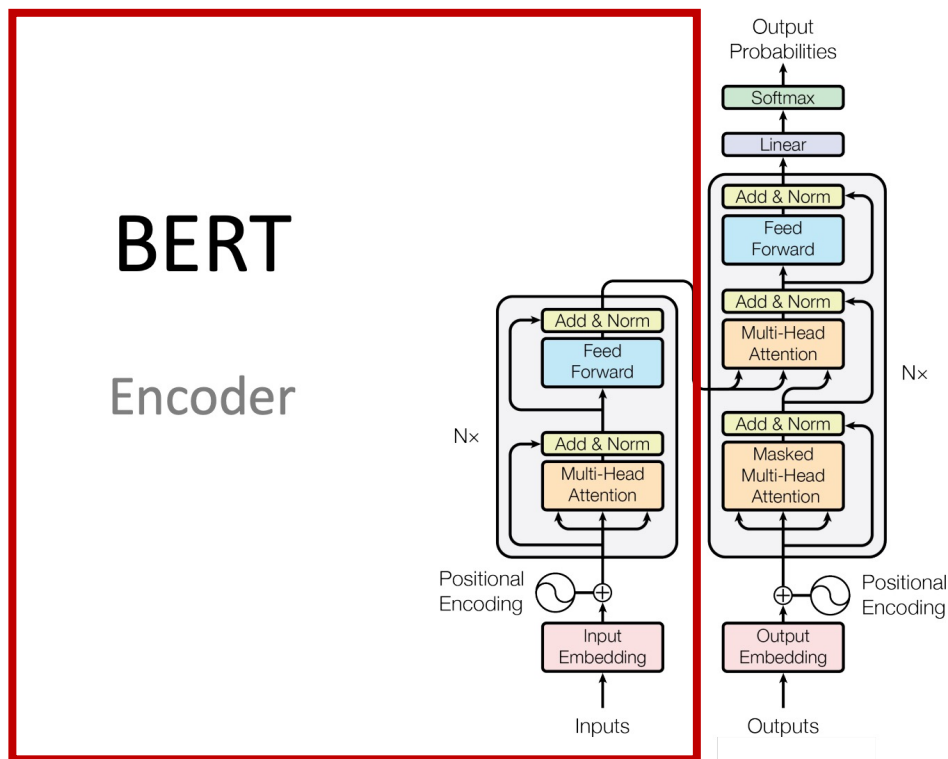
Transformer

BERT: Transformer Encoder



Transformer

BERT: Transformer Encoder



BERT Embedding

1. Input Representation Composition

BERT's input embeddings combine three components through summation:

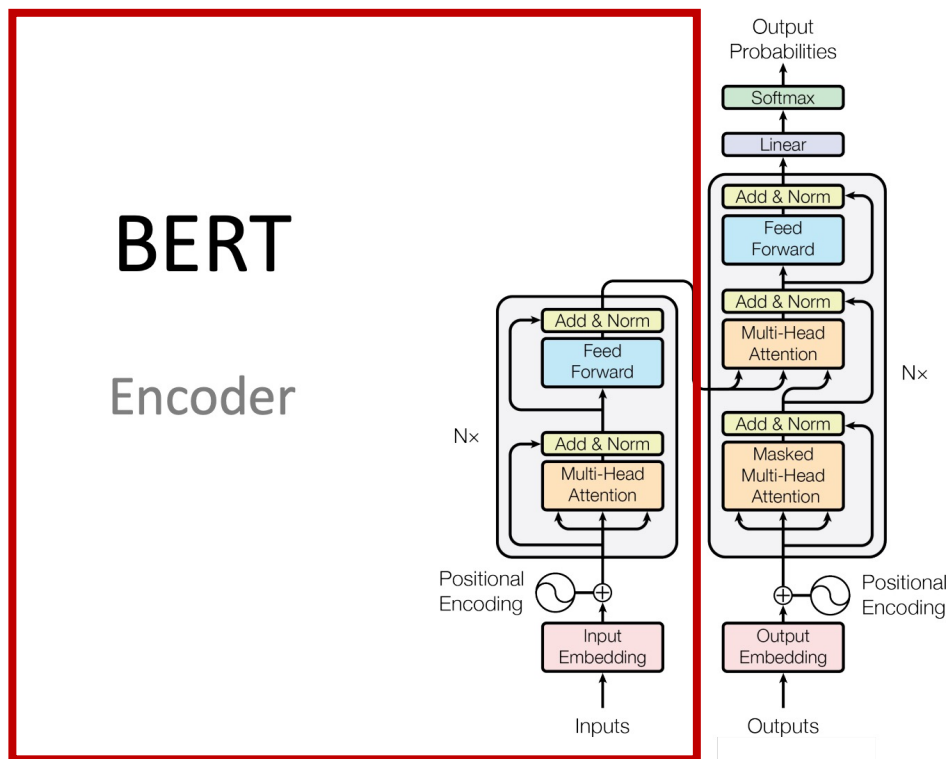
[CLS] Sentence A [SEP] Sentence B [SEP]

↓ ↓ ↓ ↓ ↓

Token Embedding + Position Embedding + Segment Embedding

Transformer

BERT: Transformer Encoder



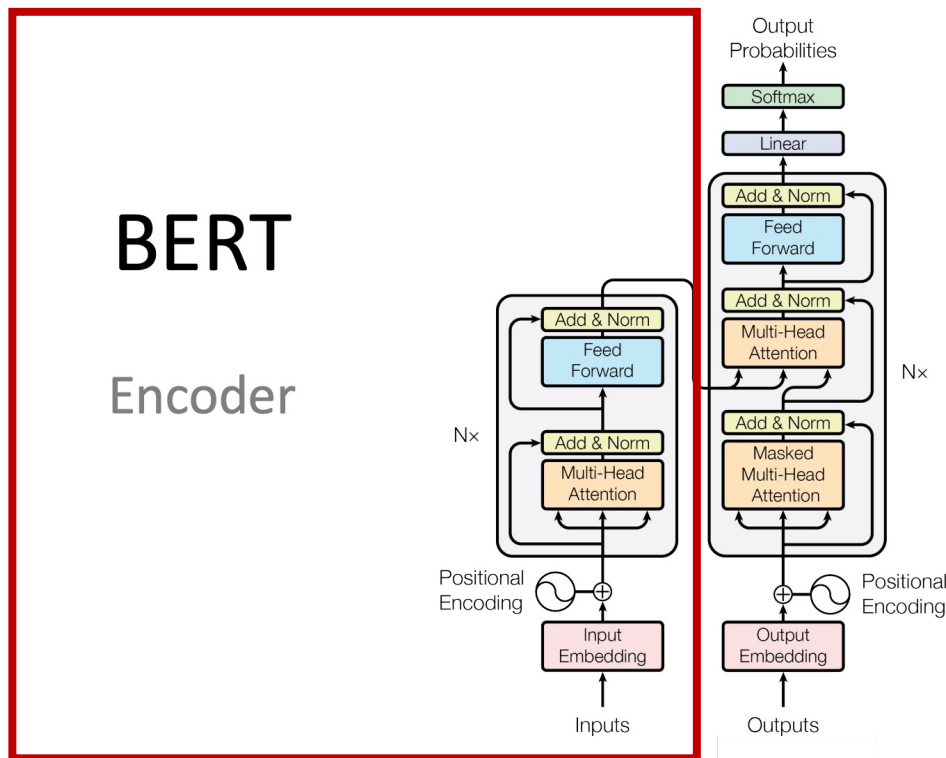
2. Token Embedding

****Implementation**:**

```
from transformers import BertModel
model = BertModel.from_pretrained('bert-base-uncased')
token_embeddings =
model.embeddings.word_embeddings(input_ids)
```

Transformer

BERT: Transformer Encoder



****Key Characteristics**:**

- WordPiece tokenization (30k vocabulary)
- OOV handling: `unfriendly` → `[ "un", "##friend", "##ly" ]`
- Special tokens:
 - `[CLS]`: Classification token
 - `[SEP]`: Separator
 - `[MASK]`: Mask token
 - `[PAD]`: Padding token

3. Positional Embedding

****Differences from Original Transformer**:**

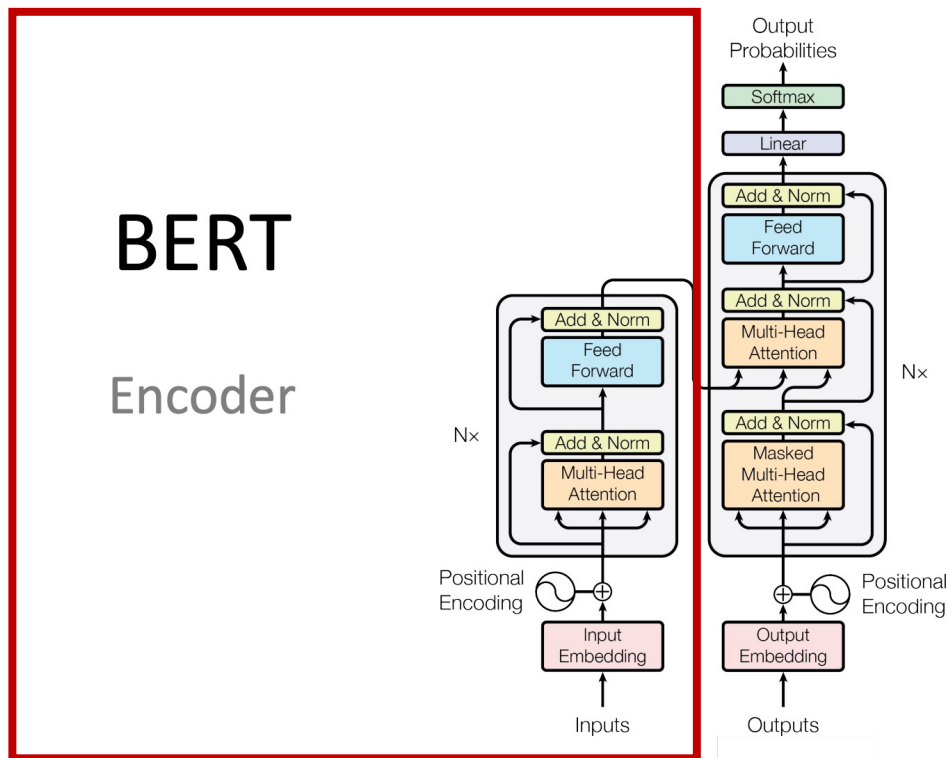
Feature	Original Transformer	BERT
Encoding	Sinusoidal	Learnable
Max Length	Fixed periodicity	512 tokens
Update	Fixed	Trainable

****Implementation**:**

```
position_embeddings =  
model.embeddings.position_embeddings(position_ids)
```

Transformer

BERT: Transformer Encoder



4. Segment Embedding

****Purpose**:**

- Distinguish sentence pairs (e.g., question and paragraph in QA)
- All zeros for single-sentence tasks

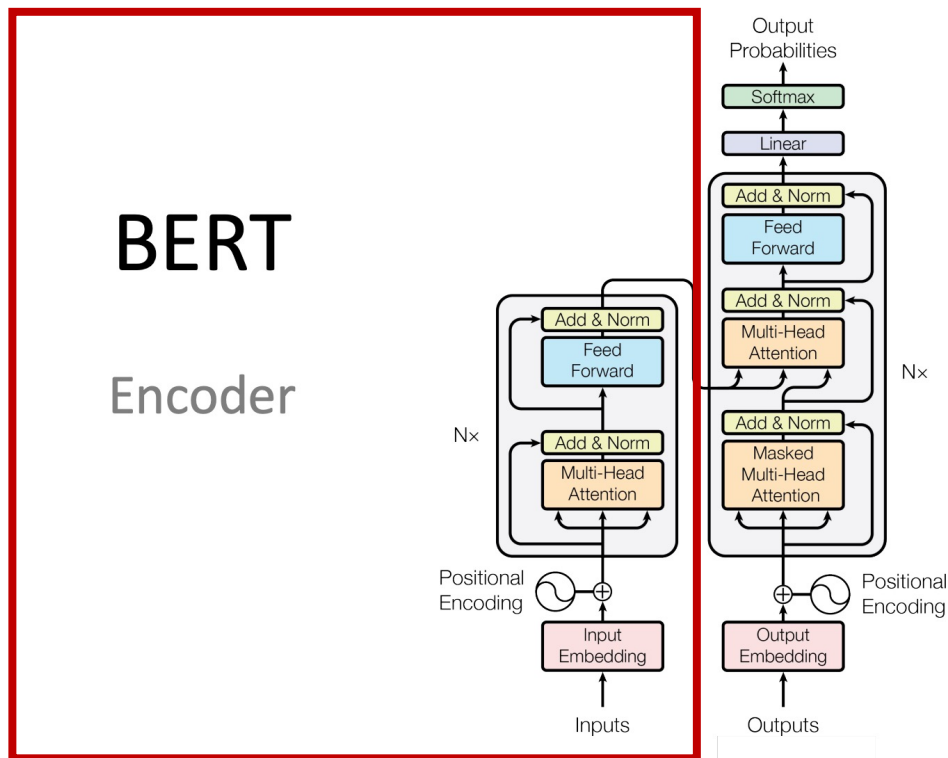
****Encoding Example**:**

Input: [CLS] How old are you? [SEP] I'm 5 years old. [SEP]

Segment IDs: 0 0 0 0 0 1 1 1 1 1 1

Transformer

BERT: Transformer Encoder



5. Composition

****Mathematical Formulation**:**

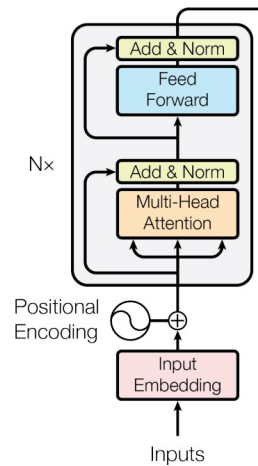
Final Embedding = TokenEmbedding + PositionEmbedding + SegmentEmbedding

Transformer

GPT: Transformer Decoder

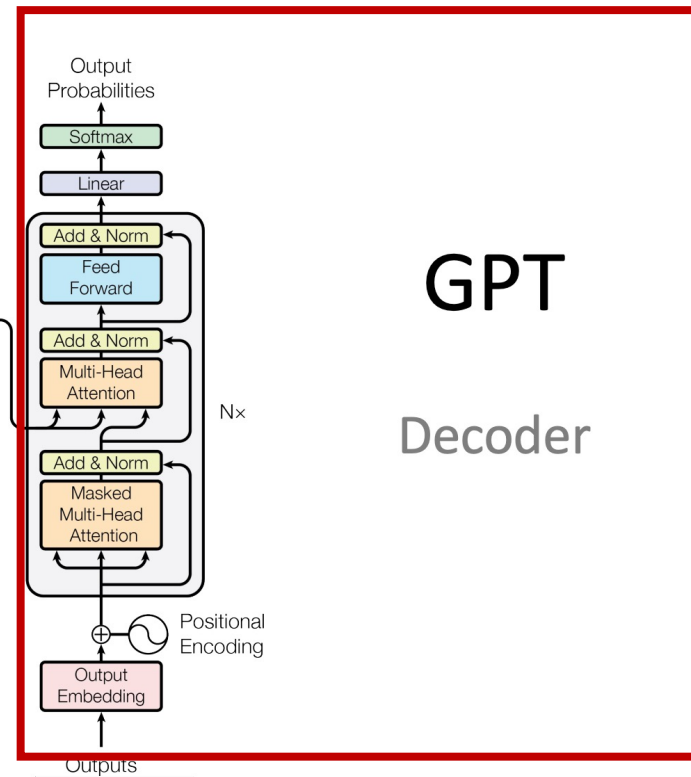
BERT

Encoder



GPT

Decoder



Transformer

GPT: Transformer Decoder

GPT Embedding Architecture

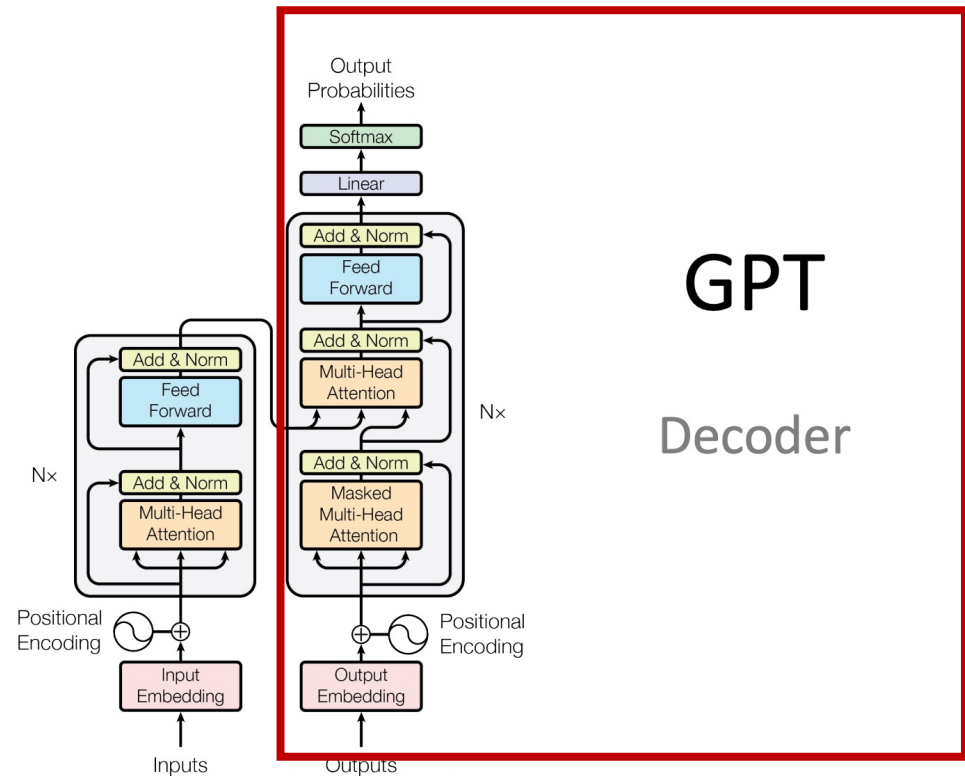
1. Input Representation Composition

GPT's input embeddings combine two components through summation:

Input: <sos> The quick brown fox jumps

↓ ↓ ↓ ↓ ↓

Token Embedding + Position Embedding



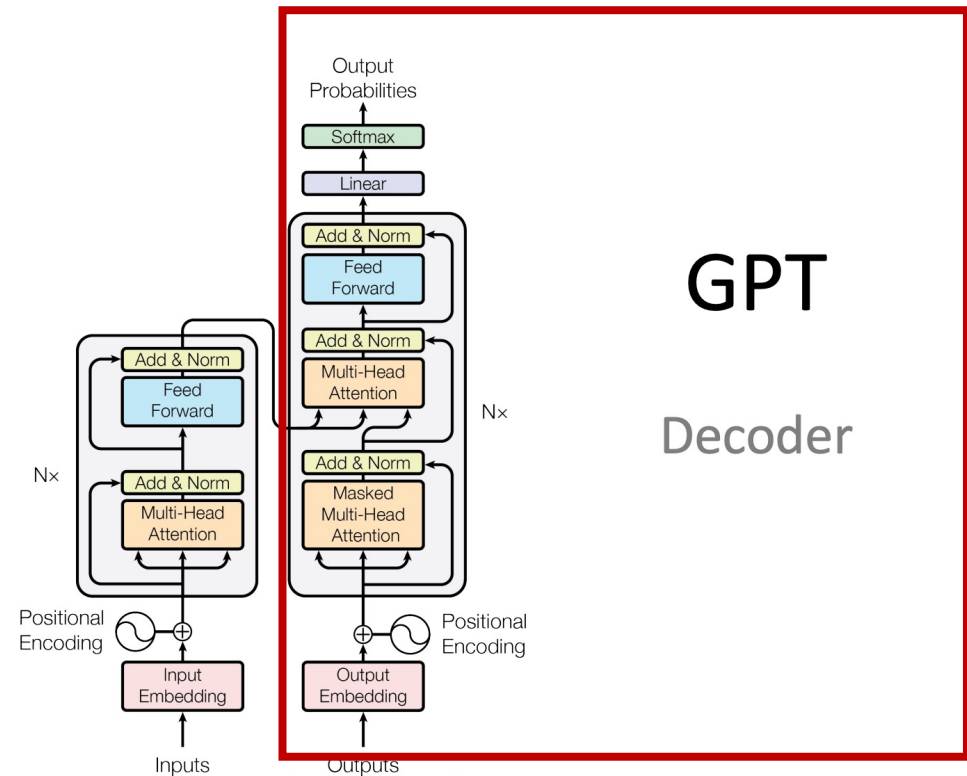
Transformer

GPT: Transformer Decoder

2. Token Embedding

****Implementation**:**

```
from transformers import GPT2Model
model = GPT2Model.from_pretrained('gpt2')
token_embeddings = model.wte(input_ids) # Word Token Embedding
```



Transformer

GPT: Transformer Decoder

****Key Characteristics**:**

- Byte-Pair Encoding (BPE) tokenization (50,257 vocab)
- Special tokens:
 - `<|endoftext|>`: Text terminator
 - No structural tokens like [CLS]/[SEP]

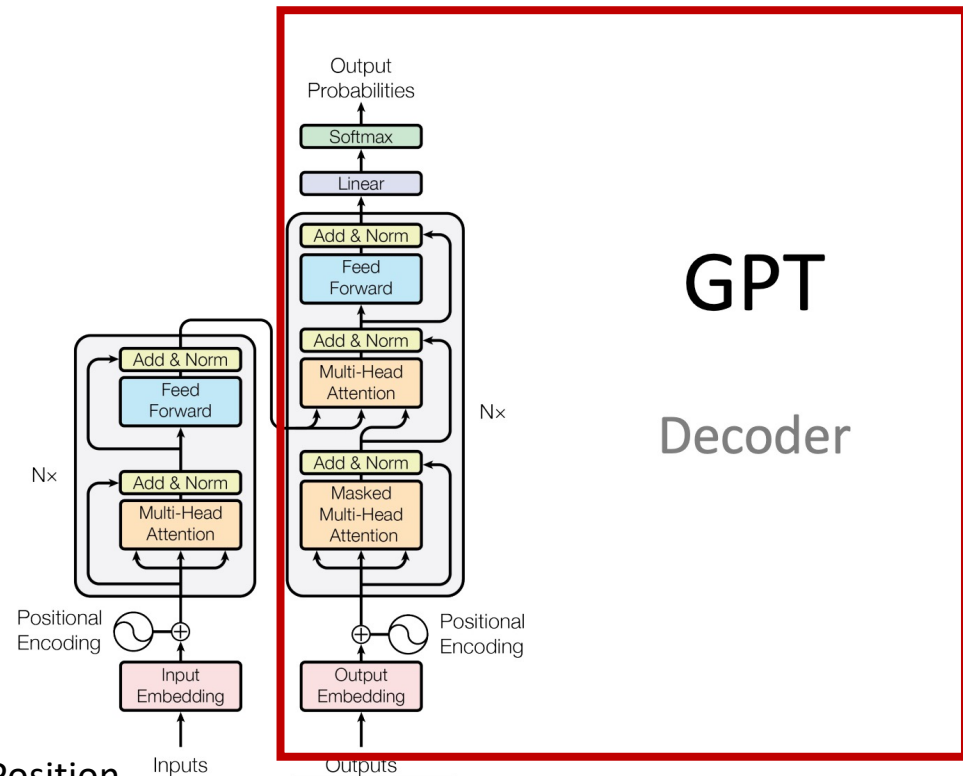
3. Positional Embedding

****Comparison with BERT**:**

Feature	BERT	GPT
Encoding	Learnable	Learnable
Max Length	512	1024 (GPT-2)
Directionality	Bi-directional	Uni-directional

****Implementation**:**

```
position_embeddings = model.wpe(position_ids) # Word Position Embedding
```



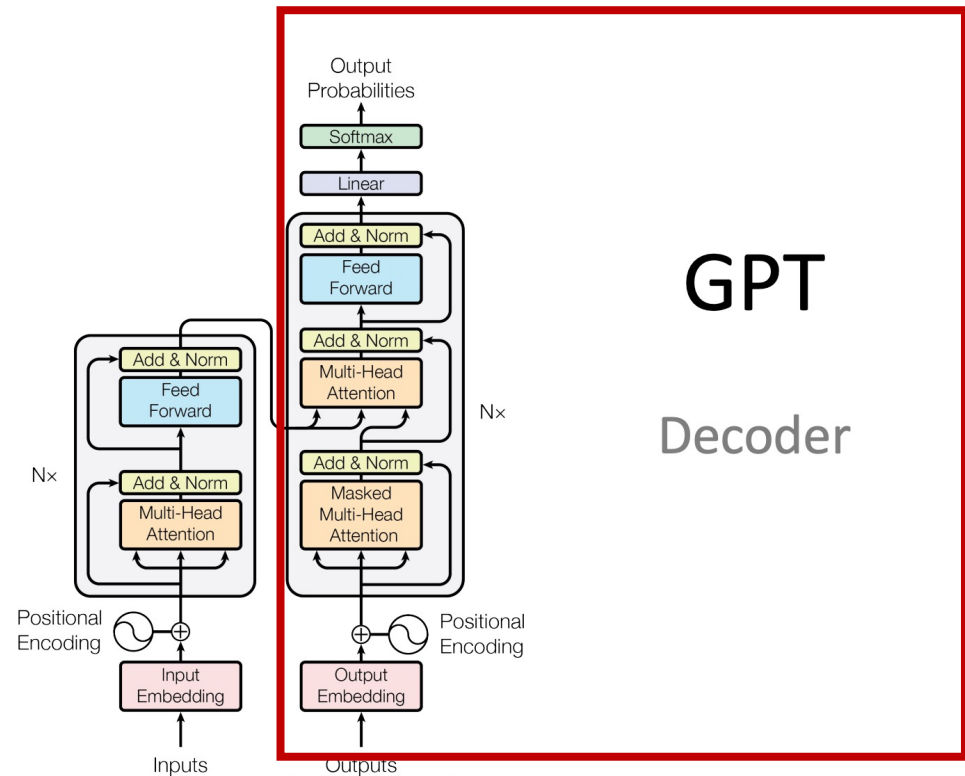
Transformer

GPT: Transformer Decoder

4. Composition

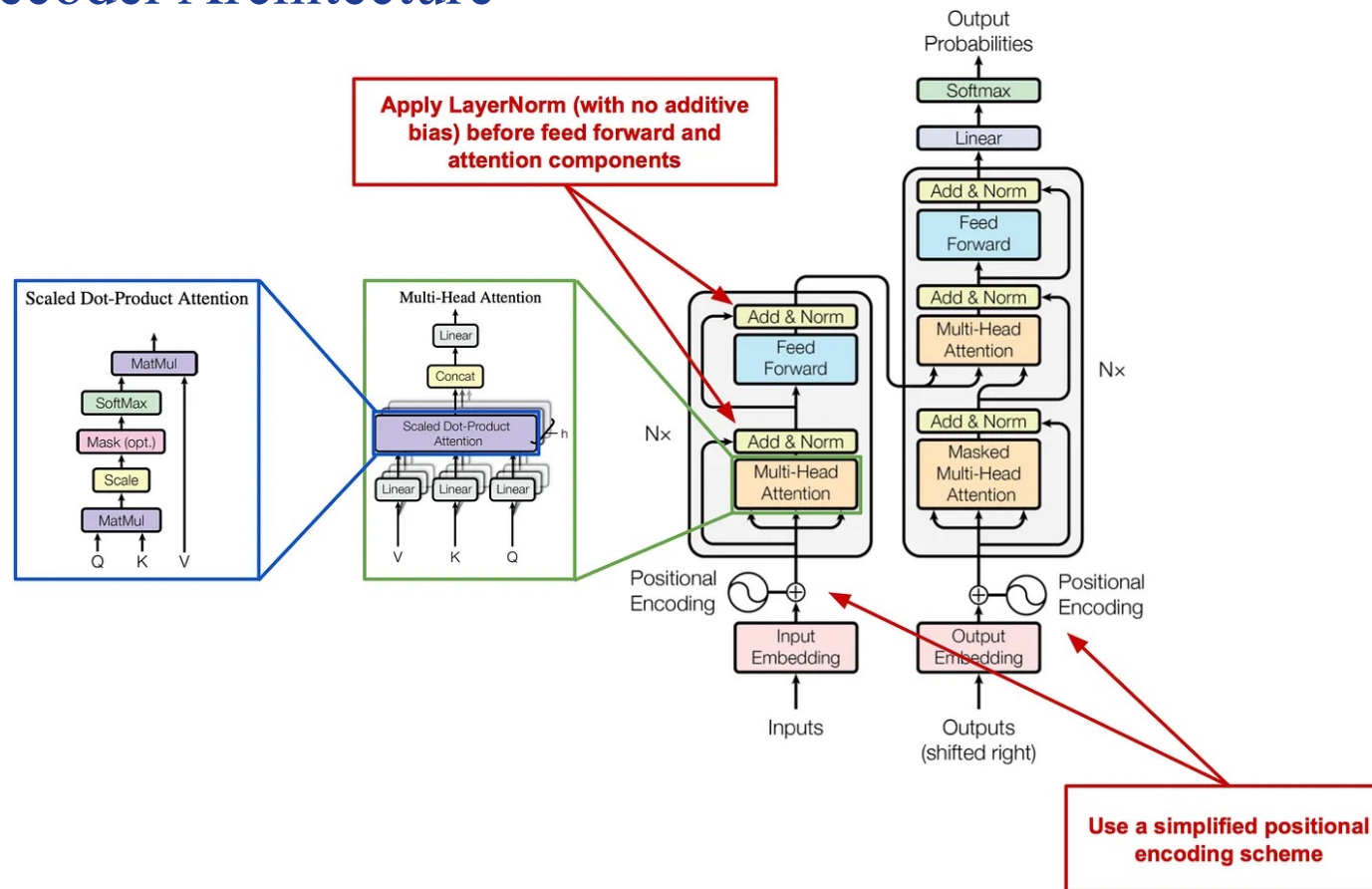
****Mathematical Formulation**:**

Final Embedding = TokenEmbedding + PositionEmbedding



Transformer

T5: Encode-decoder Architecture



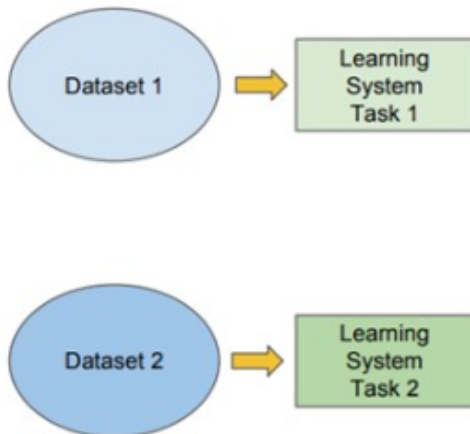
Modifications made by T5 to the encoder-decoder transformer architecture

Applications

Transfer learning?

Traditional ML

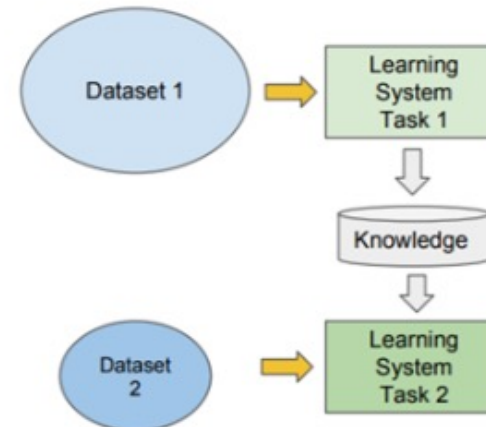
- Isolated, single task learning:
 - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



vs

Transfer Learning

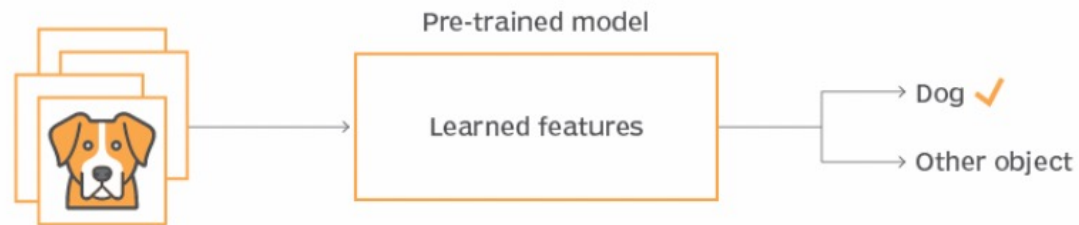
- Learning of a new tasks relies on the previous learned tasks:
 - Learning process can be faster, more accurate and/or need less training data



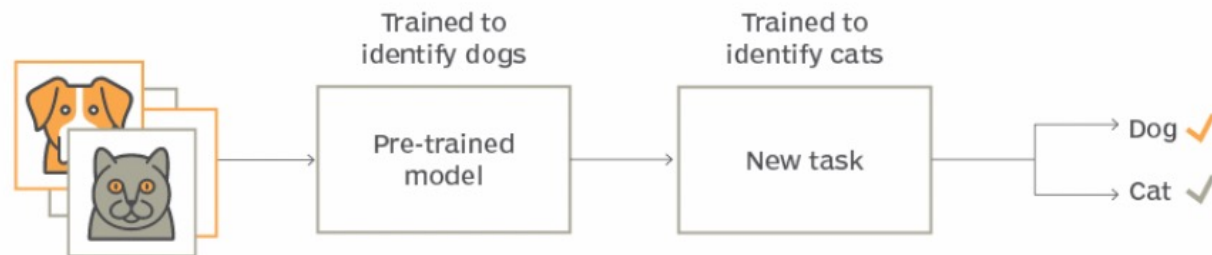
Pretrained Vision-Language Model

Transfer learning

Training from scratch

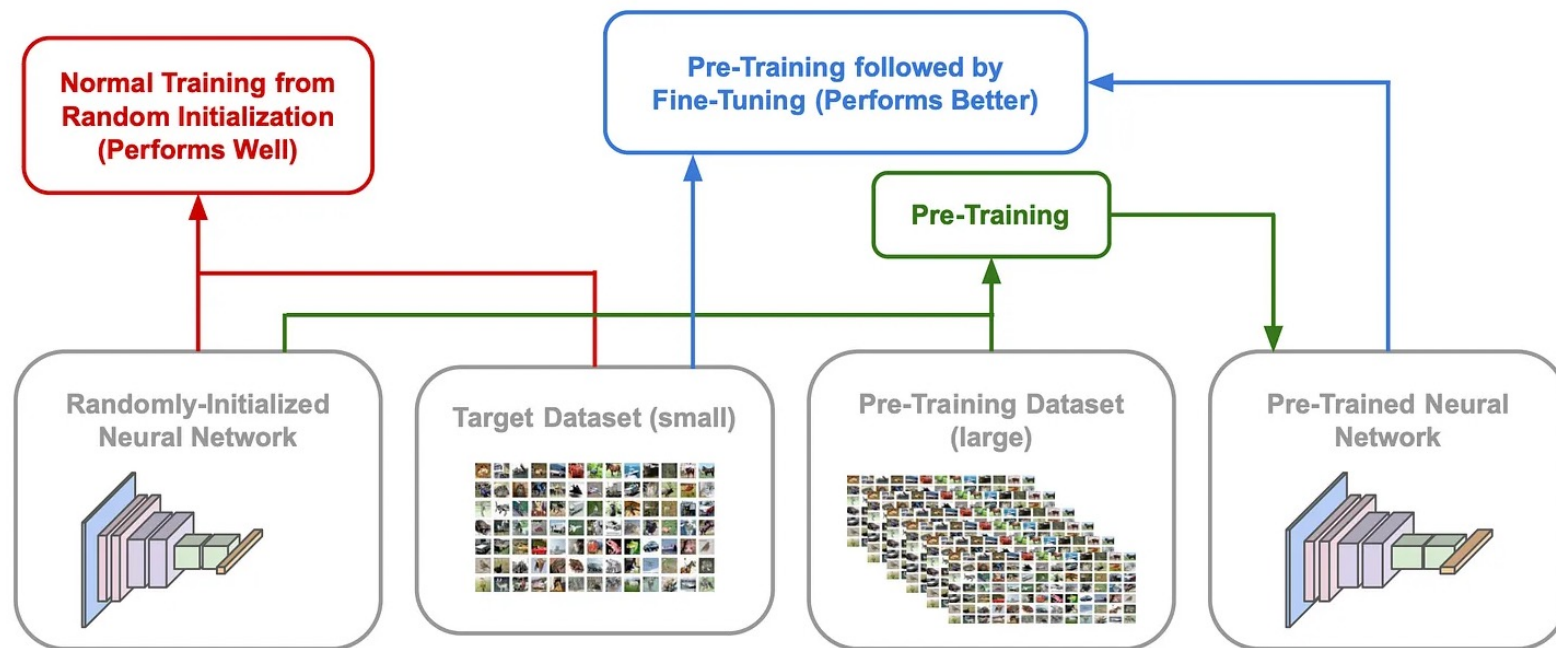


Transfer learning



Pretrained Language Model

Transfer learning



Different options for training a neural network

Pretrained Language Model

Transfer learning **from one task to another**

- Source data
 - Model trained; for example,
 - Emotion classifier
 - Hate Classifier
- Destination dataset
 - New dataset from Reddit
 - understand the domain-specific differences between the source data (used for pre-training) and your target data.
 - Example, consider differences in text length, vocabulary, or writing style.
- Collect dataset
- Divide into training, validation, and test sets.
- Model learned on source dataset will be used and our aim is to adapt this Model.
 - Freeze some or all layers of the pre-trained model (especially the earlier layers, which capture general features).
 - This ensures that the pre-trained knowledge is not overwritten.
 - Add task-specific layers (e.g., classification heads or fully connected layers) on top of the pre-trained architecture.
 - Ensure the output matches the format of your target task (e.g., a softmax layer for multi-class classification).

Pretrained Language Model

Transfer learning **from one task to another**

- Fine-tune the Model.
 - For this Set Hyperparameters: Use a smaller learning rate for the pre-trained layers to avoid drastic changes.
 - Use a higher learning rate for the new layers added for the target task.
 - the model on the target dataset, keeping an eye on overfitting (especially if the dataset is small).
 - Use techniques like early stopping, dropout, or data augmentation if needed.
- Validate Performance:
- Evaluate the model on the validation set to ensure good generalization.
 - Monitor key metrics like accuracy, precision, recall, or F1 score rly stopping, dropout, or data augmentation
 - Test the model on the holdout test set to evaluate its final performance.
 - If the model performs well, proceed to deploy it. Otherwise, consider collecting more data or revisiting fine-tuning. if needed.

Deep Learning: Recent Research

Pretrained Language Model

Transfer learning

If we want to train a neural network to solve some task, we have two basic options.

1. Training from scratch: randomly initialize your neural network and train it (in a supervised manner) on your target task.
2. Transfer learning: pre-train the network on a separate dataset, then fine-tune it (i.e., train it more) on the target task.

Pretrained Language Model

Pretraining

- Trained on large corpous
- Many layers
- Masked language modeling (MLM), next sentence prediction (NSP).

PLM	Version	Param. size (M)	Data size (GB)
BERT	small	108.3	16
	large	334.6	
RoBERTa	small	125.7	160
	large	355.4	
ALBERT	small	11.2	16
	large	206.4	
BART	small	139.4	160
	large	406.3	

Pretrained Vision-Language Model

Transfer learning

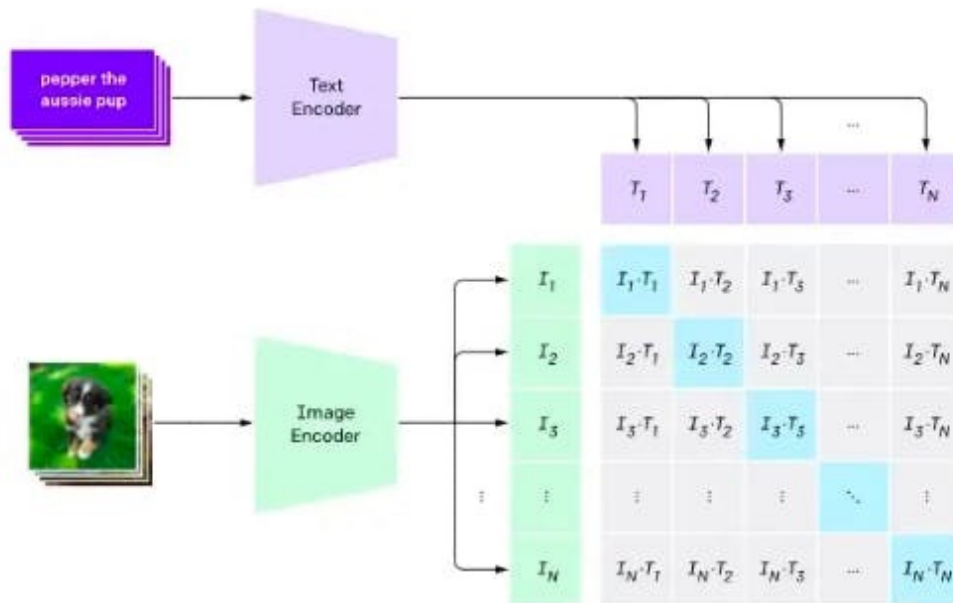
If we want to train a neural network to solve some task, we have two basic options.

1. Training from scratch: randomly initialize your neural network and train it (in a supervised manner) on your target task.
2. Transfer learning: pre-train the network on a separate dataset, then fine-tune it (i.e., train it more) on the target task.

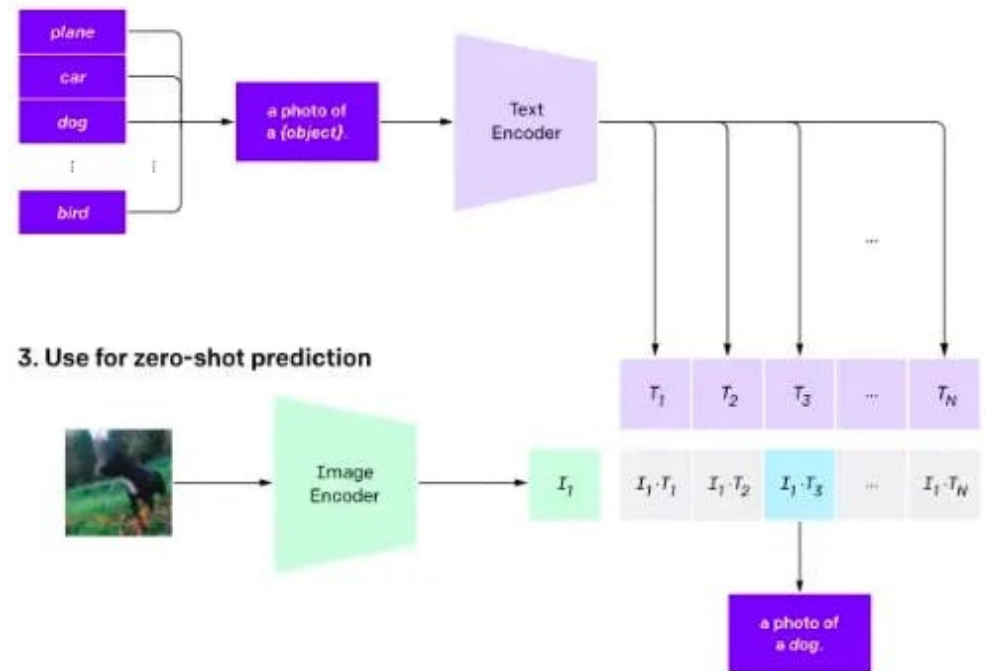
Pretrained Vision-Language Model

Pretraining: CLIP

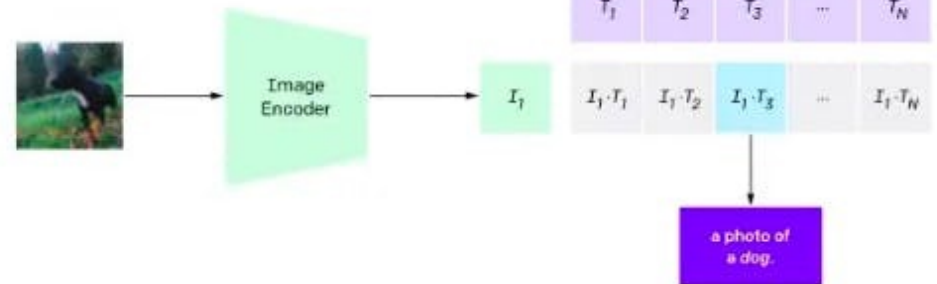
1. Contrastive pre-training



2. Create dataset classifier from label text

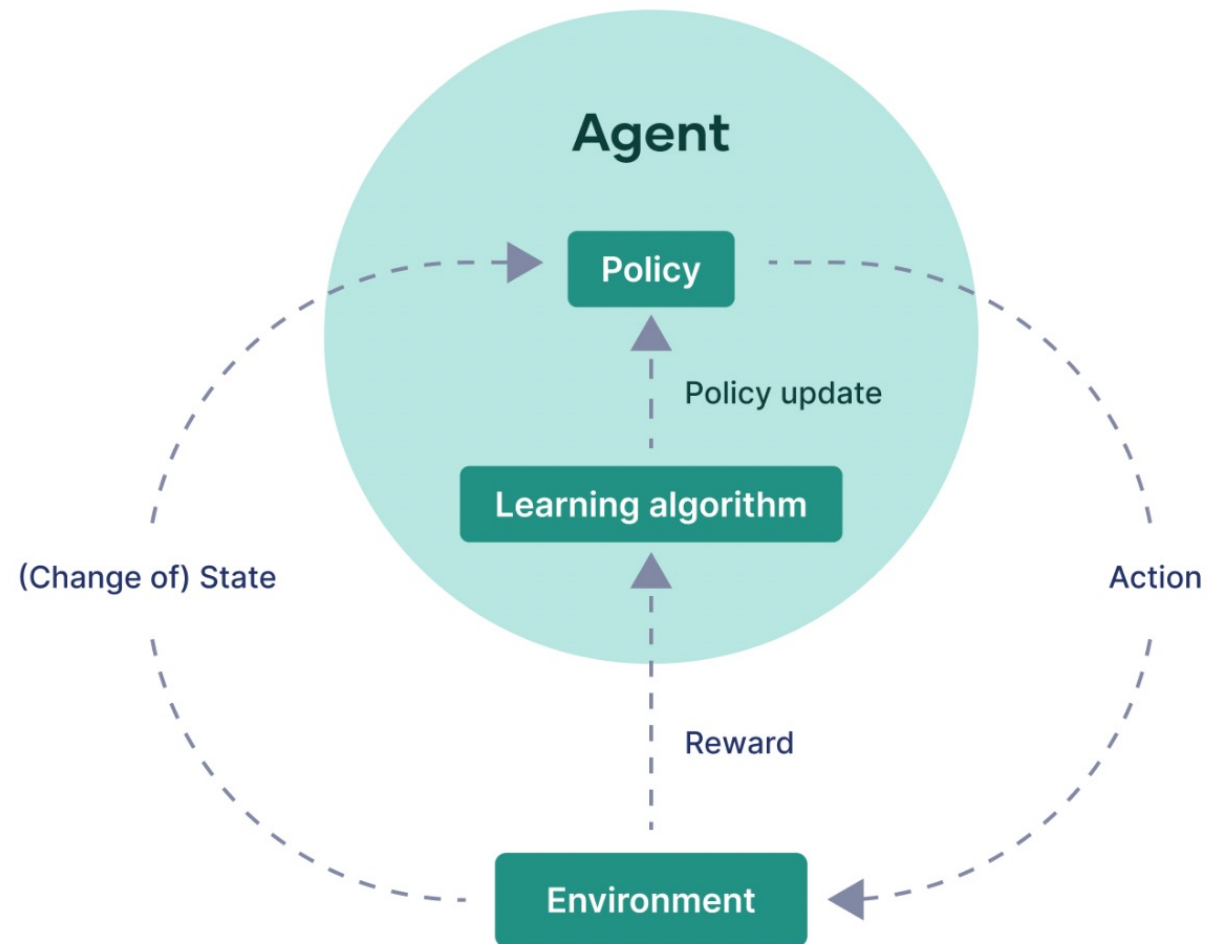


3. Use for zero-shot prediction



Reinforcement Learning and Transformers

RL basics



Reinforcement Learning and Transformers

RL basics

- Environment is everything that constitutes the game's world: the maze, the enemy (ghosts), the dots, the bonus items (fruits), the power pellets, etc.
- Agent refers to the eponymous character. The agent's goal is to “eat” all the dots in the maze, avoid the ghosts, and maximise its score.

Reinforcement Learning and Transformers

RL basics

- Action refers to moves Pac-Man can make: moving up, down, right, or left.
- States can be different situations or scenarios, such as being in a corner of the maze with no dots nearby and several ghosts approaching. The positions of Pac-Man and the ghosts, the remaining dots, and whether ghosts are in blue mode (so that Pac-Man can chase them) all constitute states.
- Reward can be positive—e.g., receiving points for eating dots—or negative—e.g., losing a “life” if caught by a ghost.

Reinforcement Learning and Transformers

RL basics

- Policy can be a rule like “if a ghost is nearby, move in the opposite direction” or “if a dot is nearby, move towards it.”
- Value can be understood as the desirability of being in a particular state. The value of the state described above, where Pac-Man is cornered by the ghosts, would be low because Pac-Man is at risk of being caught and has little opportunity to increase the score by eating dots. A state where Pac-Man eats a power pellet and can chase the ghosts would have high value because it allows Pac-Man to collect many dots, eliminate the ghosts, and increase the score.

Reinforcement Learning and Transformers

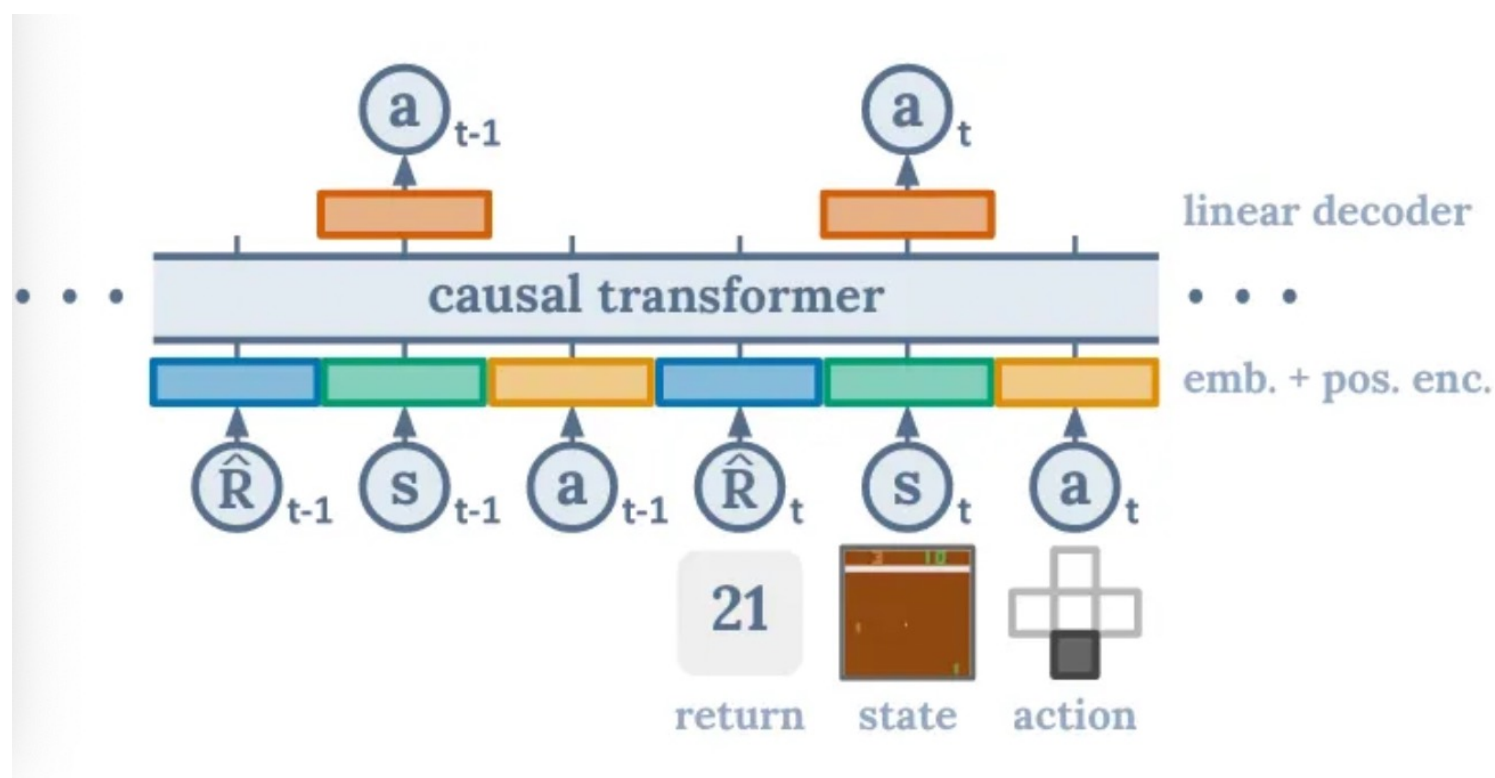
RL with Transformers

- Decision Transformer
- RLHF
- ...

Reinforcement Learning and Transformers

Decision Transformer

Frames RL as sequence modeling: $\text{State}_t \rightarrow \text{Action}_t \rightarrow \text{Reward}_t$



Reinforcement Learning and Transformers

Conflict Decision Transformer

Multi-objective classification

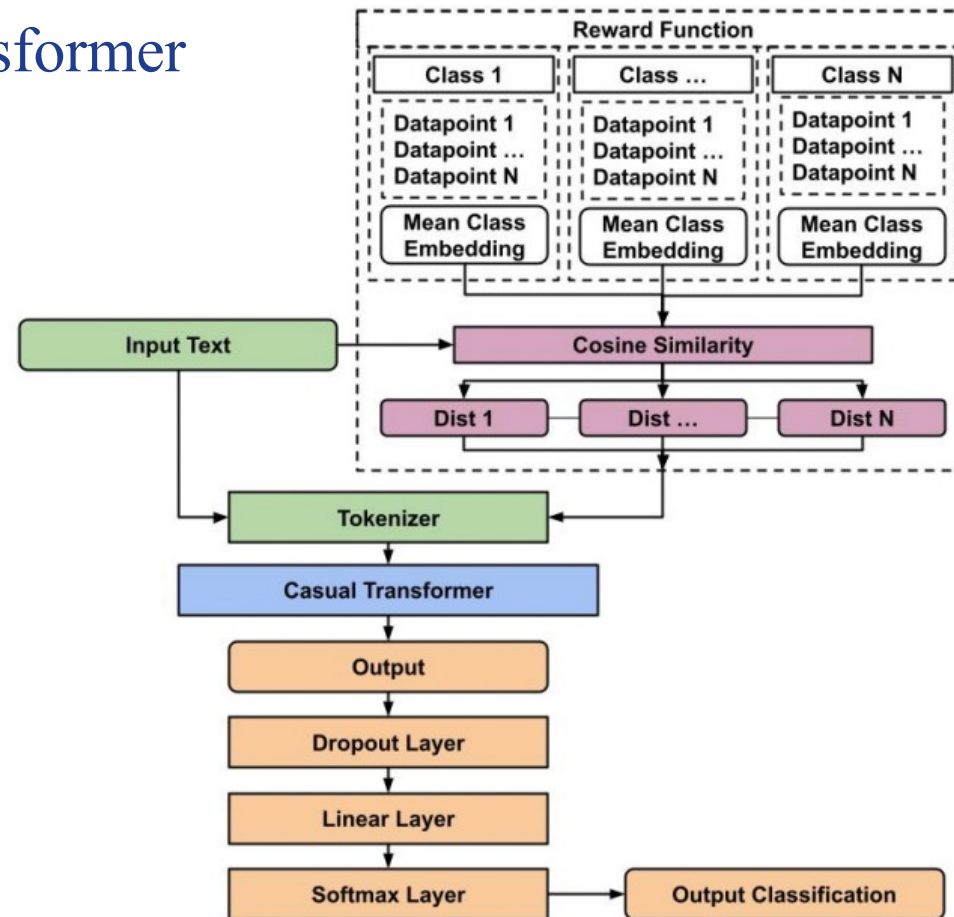
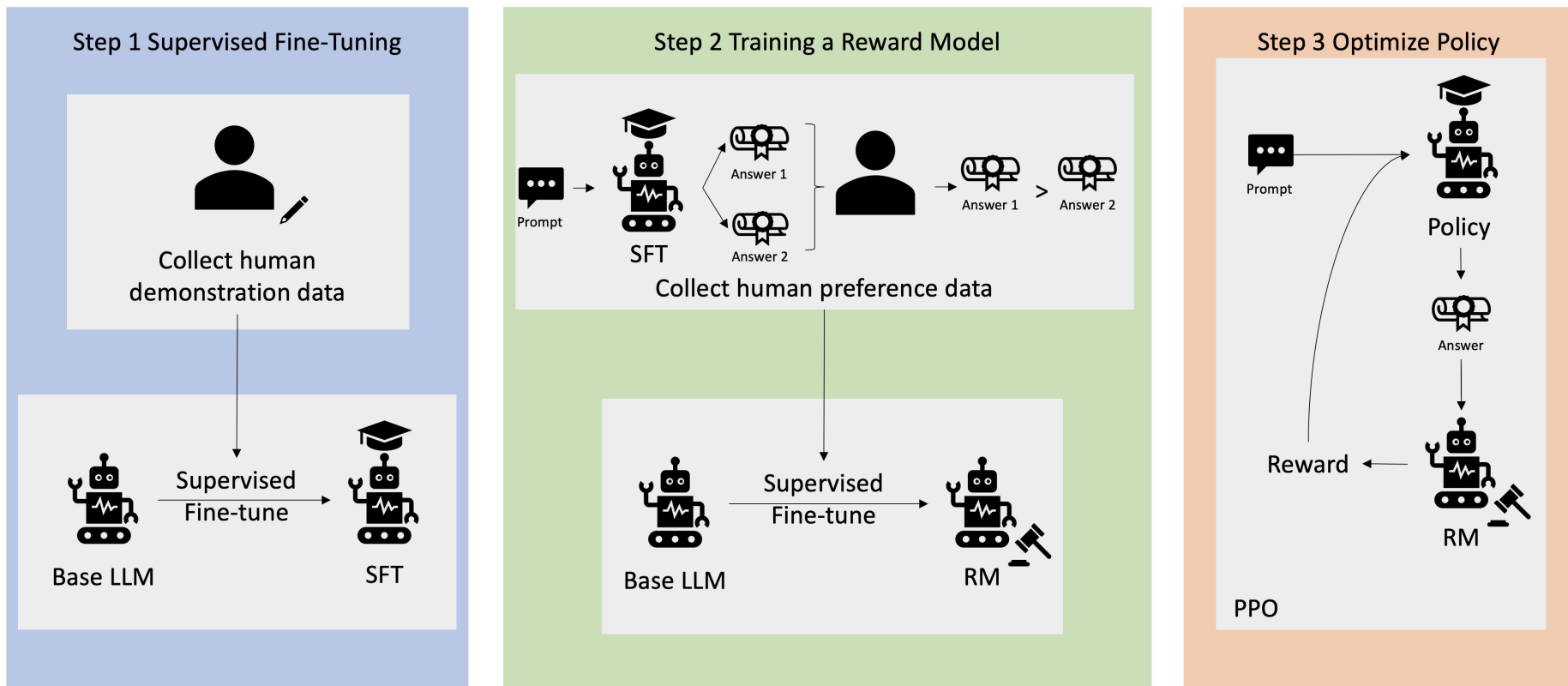


Figure 1: Model Diagram of ConflictDT.

Reinforcement Learning and Transformers

RLHF

Reinforcement learning from human feedback



Welcome Networking

research work on not only RS, LLM, MLLM ...

j.wang.9@research.gla.ac.uk